

Komunikasi Serial I2C

Materi Lengkap: Arsitektur, Protokol, Implementasi & Error Recovery

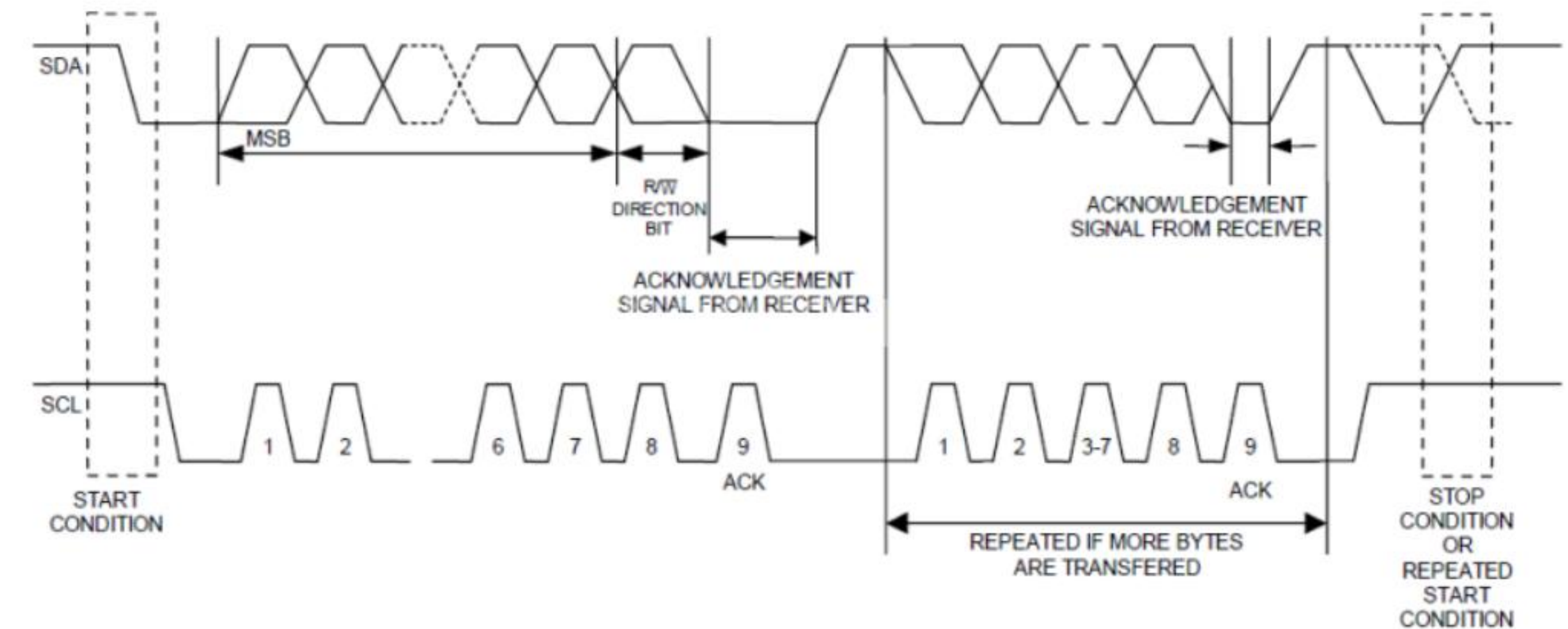
Sistem Embedded - Integrasi ESP32 & STM32

1. Arsitektur Perangkat Keras I2C

A. Bus 2-Wire: SDA dan SCL

I2C adalah protokol komunikasi sinkron biner yang hanya menggunakan dua jalur kabel (2-wire) untuk menghubungkan banyak perangkat sekaligus:

- ✓ **SDA (Serial Data):** Jalur dua arah (bidirectional) yang digunakan untuk mengirim dan menerima data bit demi bit.
- ✓ **SCL (Serial Clock):** Jalur satu arah yang dikendalikan oleh Master untuk menyelaraskan waktu (synchronization) pengiriman data antar perangkat.



1. Arsitektur Perangkat Keras I2C

B. Konfigurasi Open-Drain & Pull-Up Resistor

Semua pin SDA dan SCL pada perangkat I2C memiliki struktur internal **Open-Drain** (atau **Open-Collector**).



Prinsip Kerja:

Pin ini tidak bisa mengeluarkan tegangan HIGH secara aktif, melainkan hanya bisa menarik jalur bus ke posisi LOW (GND) atau membiarkannya mengambang (floating).



Fungsi Pull-Up Resistor:

Untuk mengembalikan kondisi bus ke nilai HIGH saat tidak ada perangkat yang menariknya ke LOW. Tanpa resistor pull-up eksternal, bus I2C tidak akan pernah bekerja.

Resistor ini biasanya bernilai **4.7 kΩ**. Tanpa ini, komunikasi akan mengalami freeze.

2. Protokol dan Struktur Frame Data

A. Sistem Pengalamatan (7-Bit vs 10-Bit Addressing)

Setiap perangkat Slave di dalam bus I2C wajib memiliki alamat unik agar Master tidak salah kirim data:

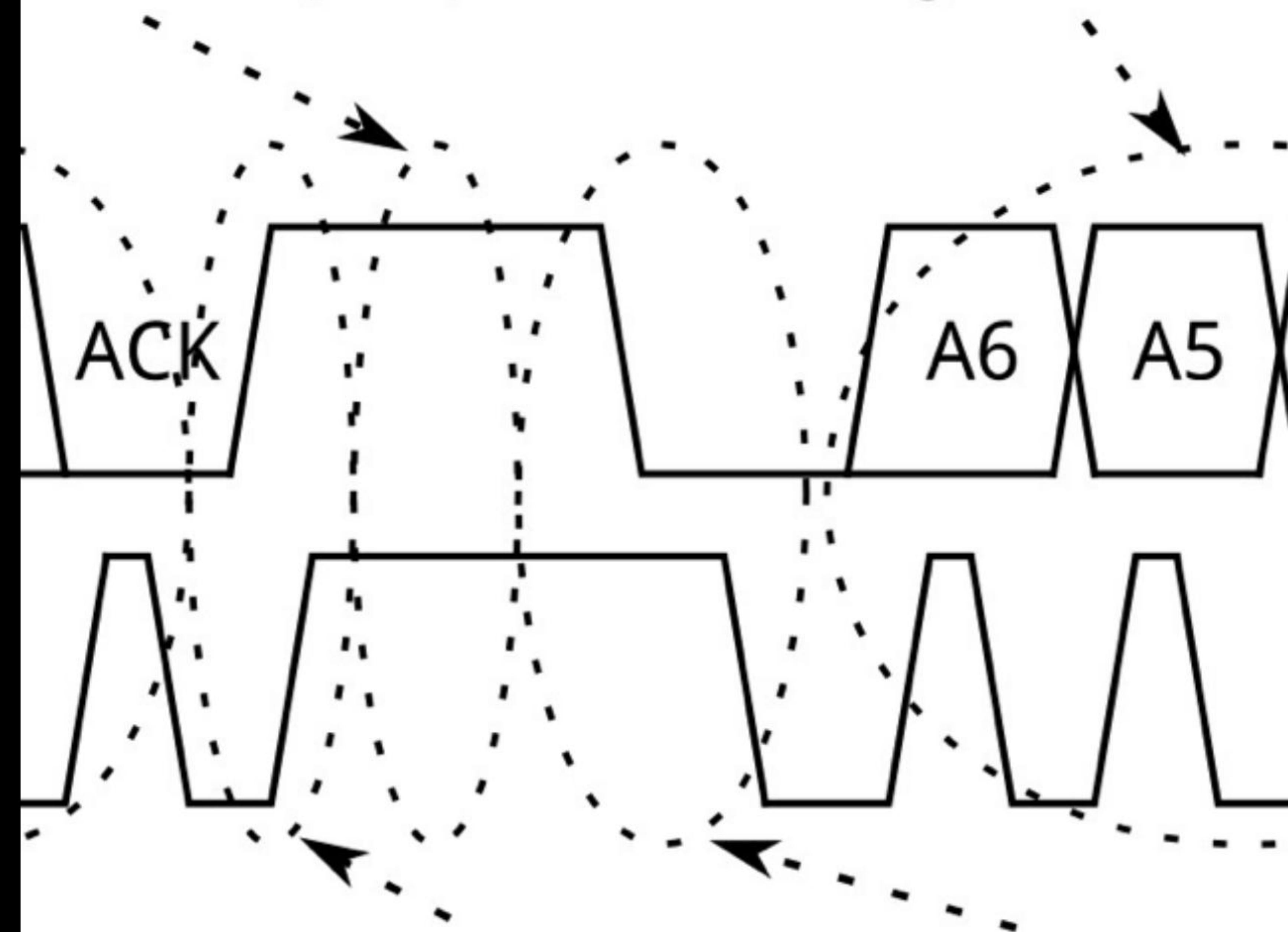
- **7-Bit Addressing (Standar):** Mendukung hingga 127 perangkat unik dalam satu bus. Bit ke-8 setelah alamat digunakan sebagai penanda arah data: 0 untuk Write (menulis) dan 1 untuk Read (membaca).
- **10-Bit Addressing (Advanced):** Digunakan jika alokasi alamat 7-bit sudah penuh, mampu menampung hingga 1024 perangkat dengan mengirimkan alamat dalam dua tahapan frame byte.

2.B Kondisi Bit Utama

- ▶ **START Condition:** Sinyal pembuka komunikasi. Terjadi ketika jalur SDA berpindah dari HIGH ke LOW saat jalur SCL masih berada di posisi HIGH.
- **STOP Condition:** Sinyal penutup komunikasi. Terjadi ketika jalur SDA berpindah dari LOW ke HIGH saat jalur SCL berada di posisi HIGH.
- ↻ **Repeated START:** Kondisi di mana Master mengulang sinyal START tanpa mengirimkan sinyal STOP terlebih dahulu. Sangat krusial untuk mengubah mode dari Write ke Read pada sensor secara instan tanpa kehilangan kendali atas bus.

idle state of the bus,
controller may assert
the bus during this period.

After the repeated start,
transfer, complete with
must begin.



frame
transfer

No stop condition
is present

Another start occurs-
this is the "repeated start"

2.C Mekanisme ACK / NACK

Setiap kali data berukuran 1 byte (8 bit) selesai dikirim, perangkat penerima harus merespons pada siklus bit ke-9 pada SCL:



ACK (Acknowledge)

Penerima menarik jalur SDA ke posisi LOW. Berarti data berhasil diterima dengan baik dan siap menerima byte berikutnya.



NACK (Not Acknowledge)

Jalur SDA dibiarkan tetap HIGH. Menandakan adanya error, alamat salah, perangkat sibuk, atau data sudah habis.

3. Performa dan Manajemen Waktu Bus

A. Klasifikasi Speed Mode

Kecepatan transfer data pada I2C diatur berdasarkan frekuensi clock (SCL) berikut:

Speed Mode	Kecepatan Maksimal	Keterangan
Standard Mode	Hingga 100 kHz	Kecepatan standar untuk sensor umum
Fast Mode	Hingga 400 kHz	Paling umum digunakan untuk sensor industri
Fast Mode Plus (Fm+)	Hingga 1 MHz	Peningkatan kecepatan transfer data
High-Speed Mode	Hingga 3.4 MHz	Membutuhkan penanganan hardware khusus

3.B Mekanisme Clock Stretching

Kondisi di mana perangkat Slave memperlambat jalurnya komunikasi karena belum selesai memproses data internal atau mengambil sampel sensor.

- 🕒 Slave akan menahan jalur SCL di posisi LOW secara paksa.
- ⌚ Hal ini memaksa Master untuk masuk ke kondisi **idle (menunggu)**.
- ▶ Master baru melanjutkan pengiriman data setelah Slave melepas jalur SCL kembali ke posisi HIGH.

4. Implementasi Hardware & Fitur Spesifik

A. Arsitektur Perangkat pada ESP32 (ESP-IDF)

-  **GPIO Matrix:** ESP32 memiliki fleksibilitas tinggi di mana periferan I2C tidak terpaku pada pin fisik tertentu. Lewat GPIO Matrix, pin SDA dan SCL bisa dipetakan ke hampir semua pin digital via software.
-  **Internal RTC & SNTP:** Digunakan untuk aplikasi data logging berbasis I2C. ESP32 mengambil waktu presisi dari internet melalui protokol SNTP (Simple Network Time Protocol), lalu menyimpannya di Internal RTC untuk mencatat waktu (timestamp) pembacaan sensor secara real-time.

4.B Arsitektur Perangkat pada STM32

-  **STM32 HAL (Hardware Abstraction Layer):** Menyediakan fungsi bawaan yang matang dan handal seperti `HAL_I2C_Master_Transmit()` dan `HAL_I2C_Slave_Receive()`.
-  **DMA (Direct Memory Access):** Aliran komunikasi I2C pada STM32 dapat dialihkan sepenuhnya ke kontroler DMA. Data dari sensor I2C akan langsung dipindahkan ke RAM secara otomatis di latar belakang tanpa membebani core CPU utama.
-  **Internal RTC:** Memiliki modul pewaktu internal mandiri berakurasi tinggi untuk sinkronisasi koordinasi data dengan Master.



5. Bus Ownership & Manajemen Multi-Master

Saat menghubungkan ESP32 dan STM32 dalam satu jaringan I2C, aturan Bus Ownership (kepemilikan bus) harus dijaga dengan ketat:

Topologi Master-Slave

ESP32 bertindak sebagai Master yang memegang kendali penuh atas SCL, sedangkan STM32 bertindak sebagai Slave dengan alamat 7-bit tertentu. Komunikasi berjalan searah demi mencegah tabrakan data.

Topologi Multi-Master

Kedua mikrokontroler bisa menjadi Master secara bergantian. Protokol I2C secara hardware akan menjalankan fungsi **Arbitrasi** jika kedua perangkat mengirim sinyal START bersamaan. Perangkat yang SDA-nya dipaksa LOW akan menang.

6. Troubleshooting & Error Recovery

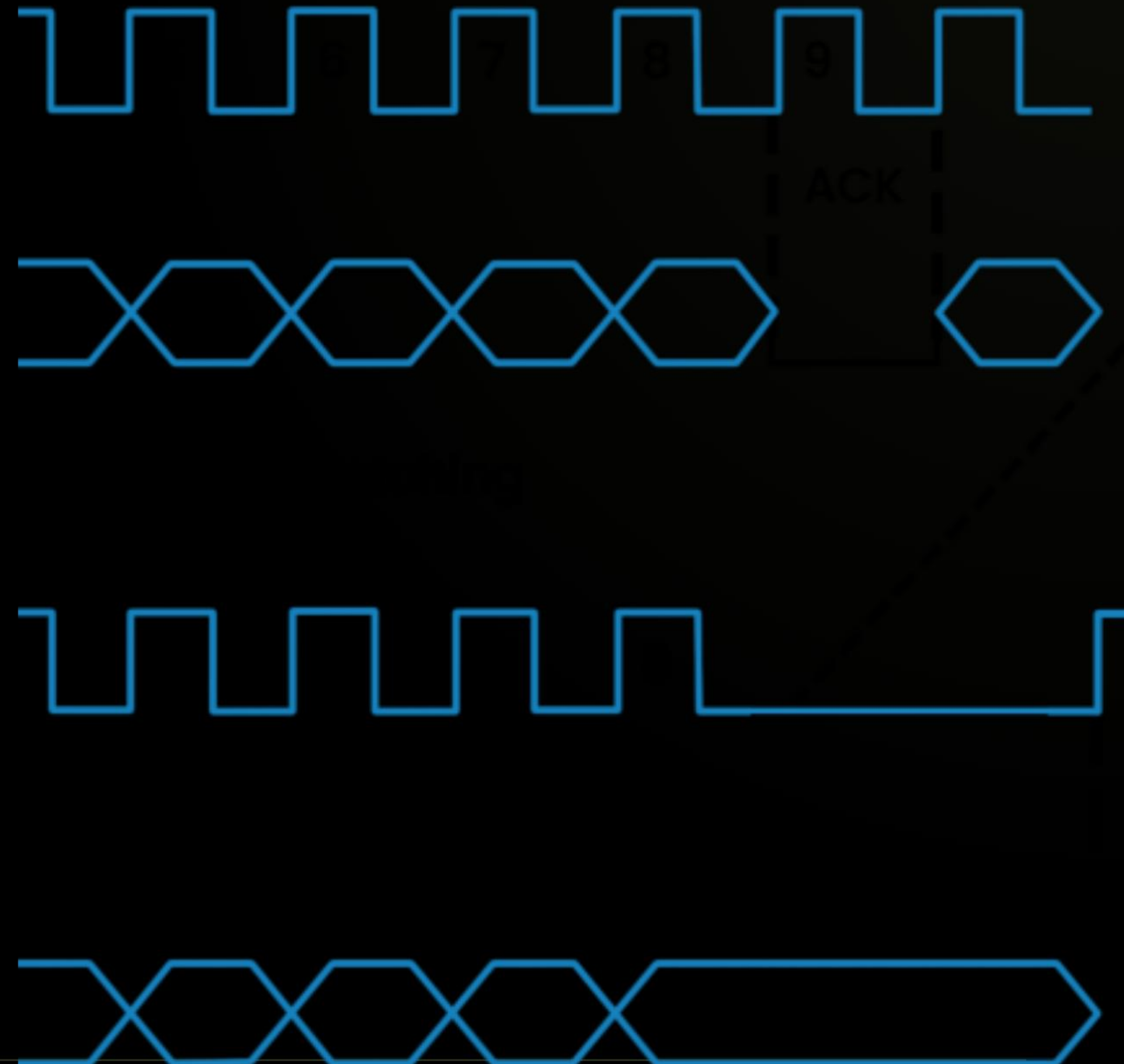
Masalah Bus Lock-Up dan Solusi "9 Pulse SCL"

Salah satu error paling fatal pada I2C adalah **Bus Lock-Up**, di mana jalur SDA tertahan di posisi LOW selamanya karena Slave mengalami crash atau terputus di tengah-tengah pengiriman data.

Penyebab: Karena SDA bernilai LOW, Master mengira bus sedang sibuk dan tidak bisa mengirim sinyal STOP. Ini mengakibatkan komunikasi lumpuh total.

Strategi Error Recovery

- 🔧 **Ubah mode pin SCL:** Ubah mode pin SCL pada Master menjadi GPIO biasa (Output Manual).
- 📡 **Kirimkan 9 kali pulsa clock:** Kirimkan pulsa (HIGH-LOW) secara manual lewat software. Ini memaksa Slave melepaskan SDA karena mengira data telah habis dibaca.
- 👉 **Sinyal STOP:** Kirimkan sinyal STOP Condition secara manual.
- 🔄 **Re-inisialisasi (Reinit):** Lakukan Reinit penuh periferan I2C untuk mengembalikan fungsi normal.



Ada Pertanyaan?

Analisis Komunikasi Serial I2C Selesai

Muhammad Dzakey Athallah

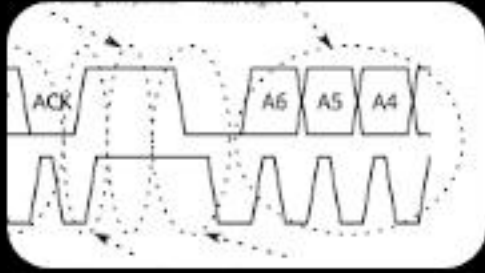
NIM 40040324620011

Prodi: Teknologi Rekayasa Otomasi - Universitas Diponegoro

Image Sources



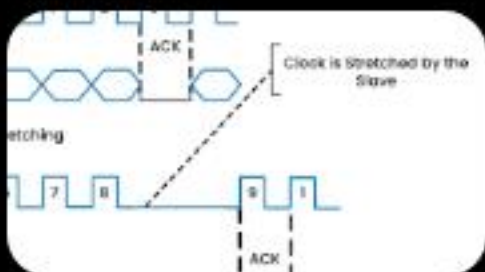
Thumbnail for <https://embedjournal.com/assets/posts/embedded/2013-05-13-two-wire-interface-i2c-protocol-in-a-nut-shell/i2c-timing-diagram.png>
Source: embedjournal.com



https://vanhunteradams.com/Protocols/I2C/repeat_start.png
Source: vanhunteradams.com



<https://community.st.com/t5/image/serverpage/image-id/61533i2143BDD2A812BF93?v=v2>
Source: community.st.com



<http://prodigytechno.com/wp-content/uploads/2022/04/Illustration-of-clock-stretching-1024x483.png>
Source: [www.prodigytechno.com](http://prodigytechno.com)
